UNIVERSITÀ DEGLI STUDI DI ROMA
TOR VERGATA



FACOLTÀ DI LETTERE E FILOSOFIA
Corso di Laurea Magistrale in
Scienze dell'informazione, della comunicazione e dell'editoria

TESI DI LAUREA IN
Sistemi di raccomandazione

Slot recommendation

Relatore:
Chiar.mo Prof.
Simone Borra

Laureando:
Andrei Carp

Correlatore:
Chiar.mo Prof.
Simone Borra

Anno Accademico 2022/2023

Indice

Introduzione....2
1.Sistema di raccomandazione....4
1.1 Tipologie....5
1.1.1 Cold Start....5
1.1.2 Collaborative filtering....7
1.1.2.1 Funk SVD....9
1.1.2.2 SVD++....10
1.1.3 Content based filtering....10
2.Slot recommendation....12
2.1 ETL....12
2.1.1 DB....12
2.2 Cosine similarity....14
2.3 Recommender....16
Conclusioni....17
Ringraziamenti....18
Bibliografia....19

Introduzione

Ad oggi internet propone una vasta gamma di siti e rispettive app di intrattenimento in ogni ambito. Siti di abbigliamento, giochi, film etc. studiano con attenzione la mole di dati che immagazzinano attraverso i cookies per poter analizzare più dettagliatamente possibile i consumatori.

Questo atto di semi-spionaggio è dovuto all'esigenza dell'azienda di poter offrire i servizi più *taylor-made* possibile, in modo da poter soddisfare il più possibile il cliente attraverso offerte di prodotti e servizi personalizzati.

Per poter rendere questo fattibile s'è pensato di implementare dei sistemi di raccomandazione. Usando algoritmi di Machine Learning tutto ciò è reso possibile.

Il motivo per cui si opta ad arrivare a suggerire agli utenti prodotti su misura è dato dall'engagement che si può ottenere attraverso suggerimenti di prodotti simili alle preferenze del cliente.

Inoltre, investire nelle risorse che permettono di conoscere il cliente ad un livello alto permette di poter avere una vasta gamma di prodotto che verrà offerta in maniera specifica e ritagliata.

Avere dei sistemi di raccomandazione efficienti porta grossi ricavi, perciò, abbiamo visto come Netflix per esempio ha creato una competizione con in palio in milione di

euro per premiare chi avrebbe superato in precisione il proprio algoritmo di raccomandazione di film.

In questa tesi si affronta il discorso di sistema di raccomandazione applicato per suggerire a ciascun cliente il gioco preferito su una piattaforma di scommesse.

1. Sistema di raccomandazione

Il mercato di prodotti digitali non smette di crescere.

Di anno in anno si vedono prodotti sempre più nuovi, sempre più all'avanguardia, ma non sono pensati in maniera tale da poter essere fruiti da un singolo consumatore.

Ogni prodotto è destinato ad un tipo preciso di consumatore e capire quale, non è di certo una missione facile. Per tale motivo, si fa leva sulla potenza di calcolo a disposizione oggi e sull'Intelligenza artificiale (AI).

Unendo algoritmi e macchina si riesce a designare in maniera chiara i gusti di una persona. Di fronte a una vasta gamma di prodotti, il consumatore ha il lavoro semplificato, perché dovrà scegliere soltanto tra prodotti che molto verosimilmente sono a suo gradimento. Ci sono diversi modi per ricostruire le preferenze del consumatore e di seguito verranno descritti alcuni dei più trattati in letteratura.

Ma se i dati su un utente non sono a sufficienza allora ci si imbatte nel problema di "cold start".

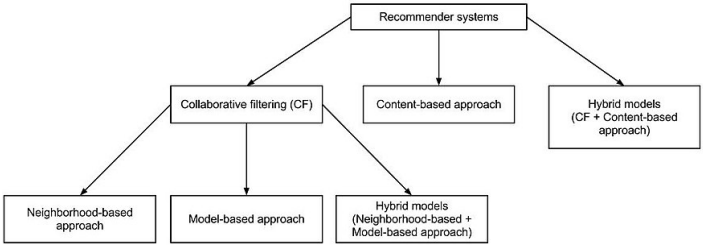


Figura 1 [*https://it.wikipedia.org/wiki/Sistema_di_raccomandazione*](https://it.wikipedia.org/wiki/Sistema_di_raccomandazione)

1.1 Tipologie

1.1.1 Cold Start

Si tratta di un problema in ambito dei sistemi informatici che si basano su modelli, che non riescono a fare inferenza su un utente oppure su un oggetto di riferimento.

Il motivo di tale problema è non si hanno dati a sufficienza; pertanto non si riesce a costruire facilmente un modello che rispecchi i gusti dell'utente.

Ci sono due modi per cercare di individuare dei pattern all'interno delle preferenze di un individuo:

- Content-based filtering (filtraggio basato sul contenuto)
- Collaborative filtering (filtraggio basato su sulla timeline dell'utente)

La partenza a freddo si intende come tre casi possibili che si verificano:

- Nuova comunità
- Nuovo articolo
- Nuovo utente

Questi tre casi rendono difficile l'uso di un filtraggio collaborativo dal momento in cui l'algoritmo si basa solo sulle interazioni dell'utente.

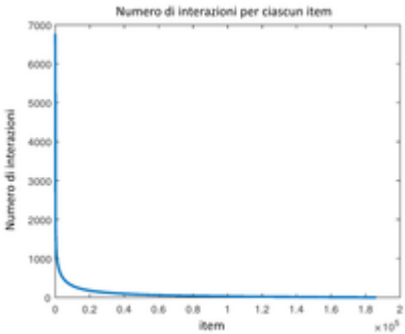


Figura 2 https://it.wikipedia.org/wiki/Cold_start

1.1.2 Collaborative filtering

"Il concetto di *collaborative filtering* è stato introdotto nel 1992 dallo staff di ricerca Xerox nell'ambito del progetto Tapestry, un sistema che consentiva agli utenti di rintracciare dei documenti basandosi su commenti lasciati da altri utenti" (David Goldberg, 1992).

Il cosiddetto filtro collaborativo permette di avere una raccomandazione attraverso la creazione di una matrice di fattorizzazione che è generata [utente x prodotto] con all'interno la valutazione che l'utente ha dato o generato per uno specifico prodotto (Shapira, 2011).



Figura 3 https://towardsdatascience.com/how-does-collaborative-filtering-work-da56ea94e331

Una volta che si ottiene la matrice, l'algoritmo tratta un utente in relazione agli altri simili che hanno generato preferenze a lui simili.

L'idea dietro a questo algoritmo è che si possano trovare preferenze simili tra utenti simili. Pertanto, sono di cruciale importanza avere come dati ciò che un utente ha visualizzato oppure votato in precedenza. Il suggerimento sarà dato dai valori più alti in termini di voto o frequenza di visualizzazione.

Questo algoritmo si trova in difficoltà per via della "cold start" precedentemente discussa.

Un algoritmo molto impiegato è la "*Matrix factorization*" o matrice di fattorizzazione. La sua notorietà è esplosa con la Netflix prize challenge. Si basa su una decomposizione

della matrice utente-item in due matrici minori il cui prodotto restituisce la matrice originaria:

$$A = U\sum V$$

dove:

U = matrice contenente fattori latenti dell'item

V = matrice contenente fattori latenti dell'utente

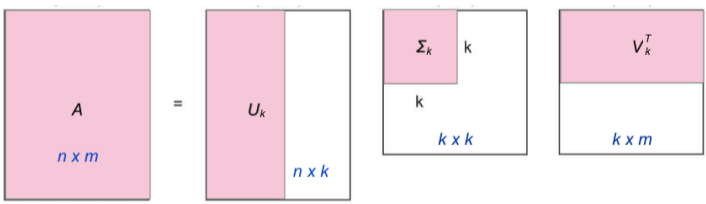


Figura 4 https://towardsdatascience.com/intro-to-recommender-system-collaborative-filtering-64a238194a26

1.1.2.1 Funk SVD

Il primo modello degno di nota è stato il Funk SVD , dal nome del creatore. Questo algoritmo minimizzava la funzione obiettivo:

$$argmin\left|\left|R - \tilde{R}\right|\right| + \alpha\left|\left|H\right|\right| + \beta||W||$$

La prima norma che è di tipo frobenius mentre le due seguenti non sono vincolate a quel tipo di norma (Paterek, 2007.).

1.1.2.2 SVD++

Il Funk SVD è un buon modello se si hanno valutazioni esplicite da parte dell'utente. Ma la sua limitazione consiste proprio in questo, essendo un'interazione user-item non riesce a spiegare bene gli apprezzamenti impliciti di un utente.

SVD++ riesce a ricavare gli apprezzamenti indiretti attraverso:

$$\tilde{r} = \mu + b_i + b_u + \sum_{f=0}^{n} H_{u,f} W_{f,i}$$

La limitazione per questo modello consiste nel non poter raccomandare nulla a un nuovo utente, il cosiddetto problema di "cold start".

Vi sono poi altri algoritmi che vano menzionati come "Asymmetric SVD" e "Deep-Learning Matrix Factorization", ma non vengono qui affrontati per il motivo di basso successo che riscontrano nel predire con esattezza le raccomandazioni efficienti. Spesso è stato dimostrato che si preferisce ottimizzare un modello semplice piuttosto che affidarsi alla rete neurale.

1.1.3 Content based filtering

L'approccio basato sui contenuti analizza le etichette, le parole chiave dell'item.

Come spiega un po' la definizione stessa si tratta di un algoritmo che prende in considerazione ciò che un utente ha comprato/letto/visto o ascoltato e lo confronta con chi compra/legge/vede o ascolta prodotti simili. Se un determinato cluster ha dei prodotti in più rispetto all'individuo analizzato, di conseguenza andrà a proporre ciò che non ha trovato nella matrice dei prodotti.

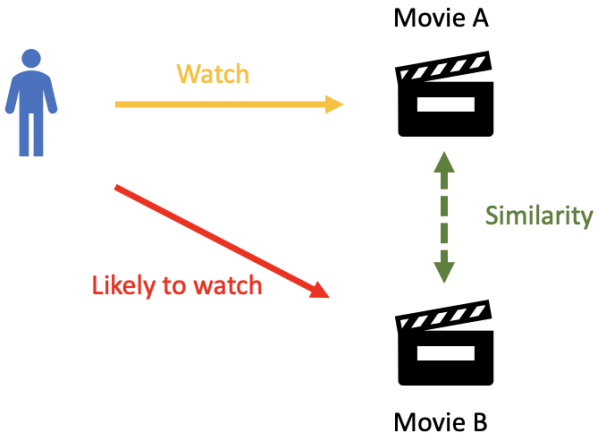


Figura 5 *https://towardsdatascience.com/essentials-of-recommendation-engines-content-based-and-collaborative-filtering-31521c964922*

2. Slot recommendation

2.1 ETL

2.1.1 DB

Lo scopo di questo lavoro è di creare una raccomandazione di giochi(slot) da poter suggerire all'utente che entra sulla piattaforma per giocare. Il database di riferimento, pertanto, sarà un'estrazione di dati dei clienti con i relativi KPI più rilevanti al fine della creazione dell'algoritmo.

Il sistema di raccomandazione dovrà suggerire giochi nuovi o simili a quelli dell'utente sulla piattaforma di slot.

Pertanto, i dati da cui si parte è la CB (customer base) che detiene le transazioni per retail e online della parte gaming degli utenti.

Il database sul quale s'è lavorato è estratto dal software Tableau ed esportato sul file di Microsoft Access Database. Il file è composto in forma matricale dove sulle righe sono presenti i clienti e sulle colonne i giochi che compongono insieme il mondo casinò.

Il db viene importato in python e convertito in DataFrame attraverso pandas. Viene fatta una serie di trasformazioni a livello di nomenclatura e anche sulla tipologia di dato che si

detiene: alcune colonne come GROSSWIN, STAKES e FREQUENCY sono trasformate in numeriche, altre come CLIENT_ID e GAME sono sottoforma di stringhe.

CLIENT_ID	GAME	FREQUENCY	GROSSWIN	STAKES
8000074	HOT BURNING WINS	1	9.20	20.40
8000074	MONSTER LEGENDS: NESSIE	2	5.00	44.00
8000074	CHRISTMAS CAROL MEGAWAYS	1	-159.90	122.50
8000074	AGE OF THE GODS NORSE: GODS AND GIANTS	1	205.38	382.88
8000074	MEGA FIRE BLAZE: BIG CIRCUS	10	-1103.75	2689.50
...	...	...	...	...
8005903	KINGS OF CRYSTALS	2	543.40	3648.80
8005903	SINBAD	2	3.55	9.60
8005903	WESTERN GOLD	1	-10.90	140.50
8005903	LUCKY SOCIAL	1	26.35	152.00
8005903	SILVER BULLET BANDIT CASH COLLECT	3	83.85	223.50

Figura 5 DB estratto da Tableau

Attraverso “drop_duplicates” di pandas si riesce a ottenere la lista giochi che compongono il db. Questa lista verrà data in pasto al loop per estrarre i features attraverso il automated browser.

Infatti, con Selenium si attiverà un automated browser che permetterà di: entrare sul sito Slot Catalog che detiene features (caratteristiche) di ciascuna slot, scrivere in automatico il nome del gioco attraverso la funzione “input” e fare webscraping della pagina.

Dato che i giochi in questioni sono 747, bisognerà fare un loop e indicargli una lista da cui prendere i nomi dei giochi che dovrà digitare autonomamente.

	Title
0	2 Hand Casino Hold’em Attributi
1	3 Card Brag (Playtech) Attributi
2	3 Devils Pinball Attributi
3	3 Fruits Win: 10 lines Attributi
4	5 Lions Megaways Attributi
..	...
738	Wolves! Wolves! Wolves! Attributi
739	Wombaroo Attributi
740	Zhao Cai Jin Bao (Playtech) Attributi
741	12 Zodiacs Attributi
742	Zombie Vegas Attributi

Figura 1 Lista giochi CB

Una volta immagazzinata l’informazione tutto viene salvato in una lista di HTML che poi verrà processata in maniera tale da renderla sottoforma di DataFrame.

Da una lista di liste contenenti l’HTML di ogni gioco si estrae la tabella di riferimento con i tag e tutte i features del gioco nonché la lista di titoli che si è riuscito a estrarre.

Una volta estrapolate le tabelle di riferimento, vengono salvate in csv per poi venire essere usate nella raccomandazione.

	Title	Provider:	Prima:	\
0	2 Hand Casino Hold'em	Evolution Gaming	01.02.2019	
1	3 Card Brag (Playtech)	Playtech	17.01.2019	
2	3 Devils Pinball	Crazy Tooth Studio	18.02.2021 [i]	
3	3 Fruits Win: 10 lines	Playson	02.05.2019	
4	5 Lions Megaways	Pragmatic Play	14.06.2021 [i]	
..	...	...	...	
738	Wolves! Wolves! Wolves!	Playtech	30.06.2018	
739	Wombaroo	Booming Games	29.10.2020	
740	Zhao Cai Jin Bao (Playtech)	Playtech	07.01.2014	
741	12 Zodiacs	Habanero	09.05.2016	
742	Zombie Vegas	Tuko Productions	01.12.2018	

	Tipo:	RTP:	Varianza:	Frequenza di vincita:	\
0	Casinò in diretta	97.84%	NaN		
1	Giochi di carte	0%	NaN		
2	Video Slot	96.7%	MED	24.39	
3	Video Slot	96.21%	MED	15.82	
4	2021-06-24	Video Slot	96.5%	HIGH	
..	...	...	...	...	
738	Video Slot	94.08%	NaN		
739	Video Slot	95.42%	HIGH	21.86[i]	
740	Video Slot	95.99% [i]	NaN		
741	Video Slot	96.1% [i]	LOW		

Figura 2 Tag e features estratti con webscraping

2.2 Cosine similarity

Tutti i dati estratti da slot catalog vengono importati per fare un'analisi "content-based" analizzando la similitudine del coseno tra due vettori, uno è quello contenente i features del gioco e l'altro è quello della matrice di tutti i giochi con tutte i features estratti.

Per analizzare due vettori si possono usare differenti metriche:

- Cosine similarity(coseno di similitudine)
- Euclidean distance(distanza Euclidea)
- Jaccard coefficient (coefficiente di Jaccard)

In questo caso s'è deciso di usare la distanza del coseno come valore di similitudine:

$$\cos\theta = \frac{A\ B}{||A||\ ||B||}$$

Dove A e B sono due vettori. Il valore della similitudine è compreso tra -1 e +1.

L’analisi condotta su VS Code usando Python inizia importando la tabella con i features che sono stati salvati grazie allo scraping precedentemente fatto.

Si trasforma i dati della matrice in un vettore grazie alla funzione TfidfVectorizer. Questo vettore viene poi usato per calcolare il coseno tra i features.

```
[(0, 1.0),
 (54, 0.8014212967471899),
 (499, 0.8014212967471899),
 (248, 0.7931541882127928),
 (652, 0.7688093651923941),
 (421, 0.7475758531329748),
 (95, 0.7276554082926578),
 (680, 0.725546274330936),
 (415, 0.7025722368896379),
 (572, 0.6970463667522779),
 (423, 0.6955858148387983),
 (97, 0.6755768962170428),
 (422, 0.6708069417525755),
 (197, 0.6700772416453075),
 (571, 0.6451948970802428),
 (622, 0.6451948970802428),
 (416, 0.6416659614859677),
 (418, 0.6271479004701502),
 (424, 0.6217580740306587),
 (159, 0.6080973725817829),
```

Figura 3 Cosine similarity tra 2 Hand Casino hold’em e tutti i giochi

Dopodiché, per confrontare la similitudine tra un gioco e gli altri giochi si usa la funzione “difflib.get_close_match”.

Questo ci permetterà di trovare i giochi che più si avvicinano a quello di riferimento.

Infine, ordinando tutti valori del coseno estraggo la top dieci dei giochi che più assomigliano.



Figura 4 Top 10 giochi da consigliare

2.3 Collaborative recommender

Per migliorare i risultati ottenuti nel paragrafo precedente si opta di analizzare la raccomandazione attraverso un filtraggio collaborativo.

Infatti, una volta salvate tutte le informazioni utili al fine del calcolo si passa a importare il file CSV estratto da Tableau che contiene i dati della nostra CB.

Il file contiene le seguenti colonne:

- CLIENT_ID
- GAME
- FREQUENCY
- GROSSWIN
- STAKES

La voce “FREQUENCY” indica quante volte nel periodo di riferimento il giocatore ha fatto una partita almeno in un giorno.

La voce “GROSSWIN” indica invece la vincita o perdita del cliente.

La voce “STAKES” indica quanto è stato raccolto in termini di puntata su un determinato gioco.

Non disponendo della variabile rating, la ricavo indirettamente utilizzando come metrica la frequenza relativa di ogni cliente. In questo caso avendo comparato le due metriche (la prima dividendo la frequenza per il totale e la seconda per massimo) s'è optato di scegliere la seconda perché restituisce risultati più parsimoniosi.

GAME	rating	num_ratings
1 MILLION MEGAWAYS BC	0.057372	2
3 DEVILS PINBALL	0.250000	1
300 SHIELDS EXTREME	0.019231	1
4 DIAMOND BLUES - MEGAWAYS	0.048077	1
5 FORTUNATOR	0.129808	2
...	...	...
WIZARDS WANT WAR!	0.009615	1
WOLF GOLD	0.038141	2
WUSHU PUNCH	0.242949	3
YATTAMAN	0.133333	1
KING BLITZ	0.364316	3

Figura 7 Rating medio di ogni gioco

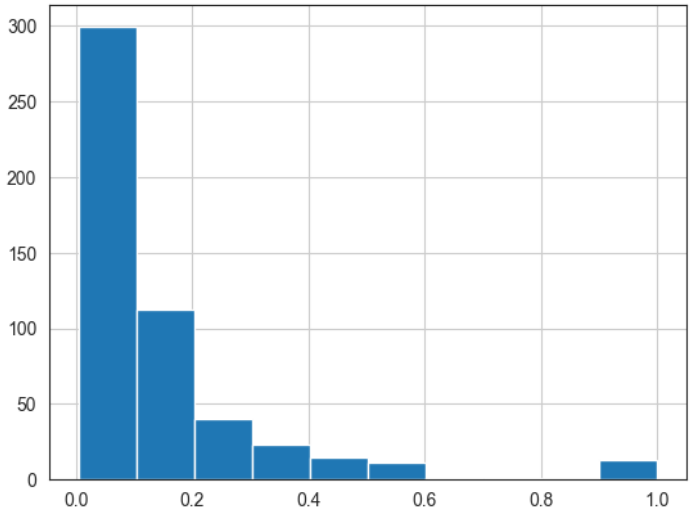


Figura 8 Istogramma del rating

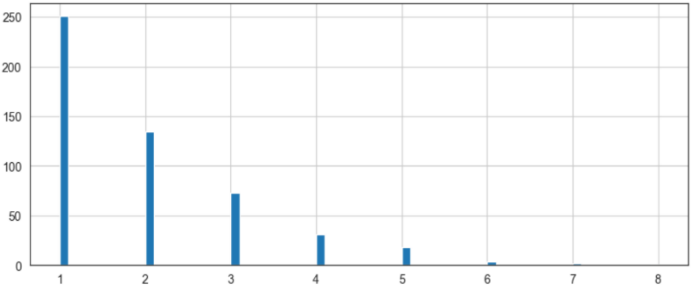


Figura 9 Istogramma numero di voti per gioco

In seguito, si crea una tabella pivot, con sulle ascisse i clienti, sulle ordinate i giochi e come valori il rating indiretto ricavato.

Si crea un benchmark che servirà per confrontare il rating: calcolando la media del rating di ogni gioco e il numero di voti.

CLIENT_ID	mean	max	min	std	max-mean	mean-min
8000074	730.167101	4771.000000	1.20	991.799730	4040.832899	728.967101
8000384	349.445879	13656.800000	2.28	1164.545833	13307.354121	347.165879
8001015	14.741667	26.750000	5.47	9.405684	12.008333	9.271667
8001357	4.200000	4.200000	4.20	NaN	0.000000	0.000000
8001553	17.400000	17.400000	17.40	NaN	0.000000	0.000000
8002051	8.800000	8.800000	8.80	NaN	0.000000	0.000000
8002415	2679.091333	39551.500000	1.00	10201.120592	36872.408667	2678.091333
8002943	11.825000	22.750000	0.90	15.450283	10.925000	10.925000
8003151	4.333333	10.100000	1.30	4.996332	5.766667	3.033333
8003421	16.324286	109.000000	0.20	40.869130	92.675714	16.124286

Figura 6 Indicatori statistici per cliente

Per vedere un gioco come si assomigli ad un altro calcolo la matrice di correlazione tra un gioco che mi interessa e la tabella pivot.

GAME	
1 MILLION MEGAWAYS BC	1.000000
ABSOLUTELY MAMMOTH! POWERPLAY JACKPOT	0.080800
ACHILLES' FORTUNE	1.000000
AGE OF GODS NORSE-KING OF ASGARD	1.000000
AGE OF THE GODS	0.985963
...	...
WILD WILD RICHES	1.000000
WITCHIN WINNIN	1.000000
WOLF GOLD	1.000000
WUSHU PUNCH	1.000000
KING BLITZ	0.117185

Figura 10 Matrice di correlazione

Infine, mi ricavo i giochi con la più alta correlazione per cui con la più alta assomiglianza tra loro.

GAME	Correlation	rating	num_ratings
SAKURA FORTUNE	1.000000	0.014161	2
RAIN BALLS	1.000000	0.104808	2
EYE OF CLEOPATRA	1.000000	0.038141	2
EYE OF ANUBIS	1.000000	0.056090	2
PYRAMID KING	1.000000	0.103205	3
...	...	...	...
SOLAR QUEEN MEGAWAYS	-0.422628	0.157532	4
BIKINI BEACH	-0.685680	0.327778	3
NAPOLI é	-0.777495	0.556548	7
FURIOUS FOUR	-0.952641	0.189899	3
FIRE BLAZE RED WIZARD	-1.000000	0.468421	2

Figura 11 Correlazione e rating di ogni gioco con Book of Egypt

Conclusioni

In conclusione, i metodi qui utilizzati sono stati piuttosto utili per estrapolare delle raccomandazioni dai titoli di gioco che i clienti hanno giocato. Entrambi gli approcci hanno riportato risultati decenti da poter essere presi in considerazione.

Possiamo sicuramente prendere in considerazione di creare nel futuro un algoritmo che impieghi entrambi.

La distanza del coseno s'è dimostrata essere una metrica abbastanza precisa. Tag e features di un item sono ovviamente buoni elementi per poter facilmente raccomandare a un utente qualcosa di simile. Infatti, nella top dieci rientrano tutti giochi del casinò come Blackjack e Poker.

In maniera verosimile, anche il secondo caso che ha usato una correlazione tra i voti medi degli utenti su determinati giochi è stato correttamente previsto nella maggior parte dei suggerimenti. Infatti analizzando le correlazioni, vediamo che i giochi più correlati a "Book of Egypt" sono tutti a tema Egitto: i nomi Anubis, Cleopatra e piramide sono tra i più correlatinie risultati finali, mentre all'opposto una slot come "Npaoli è" è la meno correlata, indicando come un gioco di cultura diversa difficilmente appaia tra quelli suggeriti.

Appendice

Install packages

```
%pip install meza
%pip install -U -q PyDrive
%pip install surprise
%pip install mdbtools
%pip install kora -q
%pip install pandas_access
%pip install pyodbc
%pip install selenium
%pip install webdriver_manager
```

Import packages

```
import pandas as pd
import numpy as np from
meza import io
# Code to read csv file into Colaboratory:
from pydrive.auth import GoogleAuth from pydrive.drive
import GoogleDrive #from google.colab import auth from
oauth2client.client import GoogleCredentials import
pyodbc
#from google.colab import data_table import time import pandas_access
as mdb import selenium from selenium import webdriver import time from
selenium.webdriver.chrome.options import Options from
selenium.webdriver.common.by import By from bs4 import BeautifulSoup
import sqlite3 from selenium.webdriver.common.service import Service
from webdriver_manager.chrome import ChromeDriverManager from
selenium.webdriver import Chrome from selenium import webdriver from
selenium.webdriver.chrome.service import Service from
webdriver_manager.chrome import ChromeDriverManager from
selenium.common.exceptions import NoSuchElementException from
selenium.webdriver.support import expected_conditions from
selenium.webdriver.support.ui import WebDriverWait from
selenium.webdriver.support.select import Select from
selenium.webdriver.support import expected_conditions as EC from
selenium.webdriver.common.keys import Keys from
sklearn.feature_extraction.text import TfidfVectorizer from
sklearn.metrics.pairwise import cosine_similarity import difflib import
matplotlib.pyplot as plt import seaborn as sns sns.set_style('white')
%matplotlib inline
```

Estrazioni features dei giochi

```
my = pd.read_excel(r"games.xlsx")
my
my_games = my["my"].drop_duplicates()
lista = list(my_games)

genre = my["Unnamed: 1"][:7555]
slot_catalog = my["Slot Catalog"][:7555]
slot_catalog = pd.DataFrame(slot_catalog)
slot_catalog["genre"] = genre  slot_catalog['my'] = my_games
my_games
current_html = []

for game in lista:


        url = "https://slotcatalog.com/it"
        chrome_options = webdriver.ChromeOptions()
```

```
        chrome_options.add_argument(r'user-data-dir=C:\Users\cesma2022\AppData\Local\Google
        chrome_options.add_argument(r'--profile-directory=Selenium')
        driver1 = Service(r"C:\Users\cesma2022\Downloads\chromedriver.exe")
        driver = webdriver.Chrome(executable_path=r"C:\Users\cesma2022\Desktop\Slot catalog\d
        driver.get(url) 24   time.sleep(2) 25   while True:
26      time.sleep(2) 27
try:
                    search  = driver.find_element(By.ID,"selsearch").send_keys(game)
                    time.sleep(2)
                    ga = posts = driver.find_element(By.XPATH,"//div[@class='tt-dataset tt-dataset-S
                    ga.click()
                    time.sleep(3)
                    current_html.append(driver.page_source)
                    break
35
                    except NoSuchElementException:
                    break
38
39  driver.quit()
40
41 current_soup =[] 42 for
i in range(747):
43  current_soup.append(BeautifulSoup(current_html[i])) 44
45          div = []
46          for s in range(747):
47          div.append(current_soup[s].find('div',{"class":"slotAttrReview"}))
48
49
50
51      table = []
52      for i in range(747):
53      try:
54      table.append(div[i].find("table")) 55   except:
56        print("No",i)
57        table
58
59      title = []
60      for i in range(747):
```

```
61          try:
62          title.append(div[i].find("h3").text) 63    except:
64            print("NO" ,i)
65            title
66            df_title = pd.DataFrame(title)
67            df_title.rename(columns={0:'Title'},inplace=True)
68            df_title.to_excel('df_title.xlsx')
69
70      df_title
71      df_title.set_index('Title',drop=True,inplace=True)
72      pd_table = pd.read_html(str(table))
73      x = pd_table[0].T
74      y = pd_table[1].T.drop(0)
75      z = x.append(y)
76      z
77      len(pd_table)
78      c = []
79      for i in range(743):
80      c.append(pd_table[i].T.drop(0))
81
82 z = z.append(c)
83 zz = pd.DataFrame(z)
84 zz.toexcel('xx.xlsx')
```

Cosine similarity

```
x = pd.read_excel('xx.xlsx')
x
x.reset_index(level=0)
x.shape
x['Title'].head()
x['Tipo:']                = x['Tipo:'].fillna('')
x['Frequenza di vincita:'] = x['Frequenza di vincita:'].fillna('')
x['Oggetti: N/A'] = x['Oggetti: N/A'].fillna('')
x['Tema: Gioco di carte'] = x['Tema: Gioco di carte'].fillna('')
x['RTP:'] = x['RTP:'].fillna('')
x['Altri : Grigio, Nero, Carta, Casinò dal vivo, Manganello, patata fritta'] =
x['Altri : Grigio, Nero, Carta, Casinò dal vivo, Mang
        x['Genere: N/A'] = x['Genere: N/A'].fillna('')
        features_ = x['Tipo:']+' '+x['Frequenza di vincita:']+' '+x['Oggetti: N/A']+' '+x['Tema: Gioco di ca
        vectorizer =  TfidfVectorizer()
        features_vector = vectorizer.fit_transform(features_)
        similarity = cosine_similarity(features_vector)
        game = input('Enter game:')
        game_list = x['Title'].tolist()
        matching_game = difflib.get_close_matches(game,game_list)
        print(matching_game)
        close_match = matching_game[0]
        close_index = x[x.Title==close_match].index.values[0]
        similarity_score = list(enumerate(similarity[close_index]))
        similarity_score
        sorted_similarities = sorted(similarity_score,key=lambda x:x[1],reverse=True)
        (sorted_similarities)
        i=1
        for movie in sorted_similarities:
        indexx = movie[0]
        title_index = x[x.index==indexx]['Title'].values[0]  31   if i<11:
          print(i,'.',title_index)
          i+=1
```

```
Sample recommender
#import data with io
rec = io.read_mdb('/content/drive/MyDrive/Colab Notebooks/Dataset.mdb')
df_mdb = pd.DataFrame(rec)

df_mdb = pd.read_sql('select * from Data',conn)
df_mdb = df_mdb.rename(columns={"Freq (AGG)":"FREQUENCY","GROSSWIN (SOMMA)":"GROSSWI
df_mdb = df_mdb.sort_values("CLIENT_ID")
df_mdb
sample = df_mdb[:1000]
sample = sample.set_index("CLIENT_ID")
sample

x = sample.groupby(sample.index)["STAKES"].agg(["mean","max","min","std"])

x["max-mean"] = x["max"] - x["mean"]
x["mean-min"] =   x["mean"] - x["min"]

        x
        sample["rating"] = sample["FREQUENCY"]/(sample.groupby("CLIENT_ID")["FREQUENCY"]).ma
        pivot_sample = sample.pivot(columns="GAME",values="rating")
        pivot_sample
        avg_rating  = sample.groupby("GAME")["rating"].mean().sort_values(ascending=False)
        sample.groupby("GAME")["rating"].count().sort_values(ascending=False)
        ratings = pd.DataFrame(sample.groupby("GAME")["rating"].mean())
        ratings["num_ratings"] = pd.DataFrame(sample.groupby("GAME")["rating"].count())
plt.figure(figsize=(10,4))
ratings['num_ratings'].hist(bins=70)
ratings["rating"].hist()
book_of_egypt_rating= pivot_sample["BOOK OF EGYPT"]
similar_to_book_of_egypt = pivot_sample.corrwith(book_of_egypt_rating)
corr_book_of_egypt = pd.DataFrame(similar_to_book_of_egypt)
corr_book_of_egypt.dropna(inplace=True)
```

Bibliografia

David Goldberg, D. N. (1992). Using collaborative filtering to weave an information tapestry. *ACM Press*.

F.O. Isinkaye, Y. F. (2015). *Recommendation systems: Principles, methods and evaluation.*

Paterek, A. (2007.). Improving regularized singular value decomposition for collaborative filtering (PDF). *Proceedings of KDD cup and workshop*.

Shapira, F. R. (2011). *Recommender Systems.* Springer.

Steck, P. A., Pershouse, M. A., Jasser, S. A., Yung, W. K., & Tavtigian, S. V. (2011). *EIN TUMORSUPPRESSOR MIT DER BEZEICHNUNG TS10Q23.3*. Tratto il giorno 1 14, 2023 da https://lens.org/199-096-428-235-027